

LLMs + Formal Methods

What I Understand

1. Where I Started

Why do we need LLM plus Formal Methods in the first place?

My starting point was fairly simple. LLMs are already very good at language, coding, planning, reasoning, and tool use. So why add formal methods at all?

The answer I arrived at is that capability and guarantee are not the same thing. An LLM can give an answer that looks completely convincing and still be wrong. A stronger model can make that happen less often, but it does not automatically turn a probabilistic system into a system with a machine checked correctness guarantee.

Formal methods give us the other side of the equation. They let us describe properties precisely and then use a solver, model checker, or theorem prover to check those properties. The interesting combination is therefore not simply making the LLM smarter. It is letting the LLM explore and generate while using formal methods to decide whether the result is acceptable.

Is the real goal to make the LLM correct, or to build a system where incorrect output cannot simply be accepted?

I think the second question is much more useful. I do not necessarily need the LLM itself to become a perfect reasoner. I need the overall system to have a reliable acceptance boundary.

2. Better Models Are Not the Same as Guaranteed Models

If frontier models are already much better at reasoning, why not just keep making the models larger?

This became one of the main things I wanted to separate. Larger and better models clearly improve performance. They can use more inference time, better training, better reasoning strategies, retrieval, tools, and other mechanisms. But even very high accuracy is different from a formal guarantee.

That distinction is especially important when one wrong answer has a serious consequence. If a model writes a bad user interface, the result may be annoying. If it writes security logic, kernel code, a protocol, or safety critical control software, the same kind of mistake can become much more serious.

Can a small model plus formal verification reach the reliability of a much larger model without verification? Maybe a Benchmark

This is one of the strongest experiments I see coming out of the discussion. Instead of only asking which model gets the highest benchmark score, I want to ask whether formal verification can compensate for some of the capability gap between a small model and a frontier model.

3. I Do Not Think All LLM Outputs Need the Same Verification

Can we really send every LLM generated artifact through the strongest formal verification pipeline?

I do not think that is realistic. A React component, a payment calculation, an authentication mechanism, a distributed protocol, and a safety controller do not have the same correctness requirements.

So I started thinking less in terms of whether code is important and more in terms of what correctness properties the artifact actually needs. (Some sort of segregation based on "Importance of the code")

Artifact	What I care about	Possible verification
UI code	Basic functional behaviour	Tests, types, static analysis
Business logic	Constraints and invariants	Contracts, SMT, tests
Security logic	Security properties	Static analysis, formal verification
Protocol	State behaviour, safety, liveness	Model checking
Critical algorithm	Functional correctness	Theorem proving, refinement
Safety controller	Safety and reachability	Model checking

This gives me a possible adaptive verification architecture. The system would first understand the artifact and its assurance requirement. It would then choose an appropriate verification mechanism instead of blindly applying the most expensive method every time.

Can the system itself decide how much verification is enough?

That seems like a useful research direction. But that is for some later when we actually find a solid verification system on different levels.

4. First Classify the High Stakes Use Cases

Where are LLMs involved in a way that makes a mistake genuinely expensive?

The first classification should be about the LLM's role. There is a major difference between an LLM that is only an interface to an already deterministic symbolic system and an LLM that is itself generating or reasoning toward the final output.

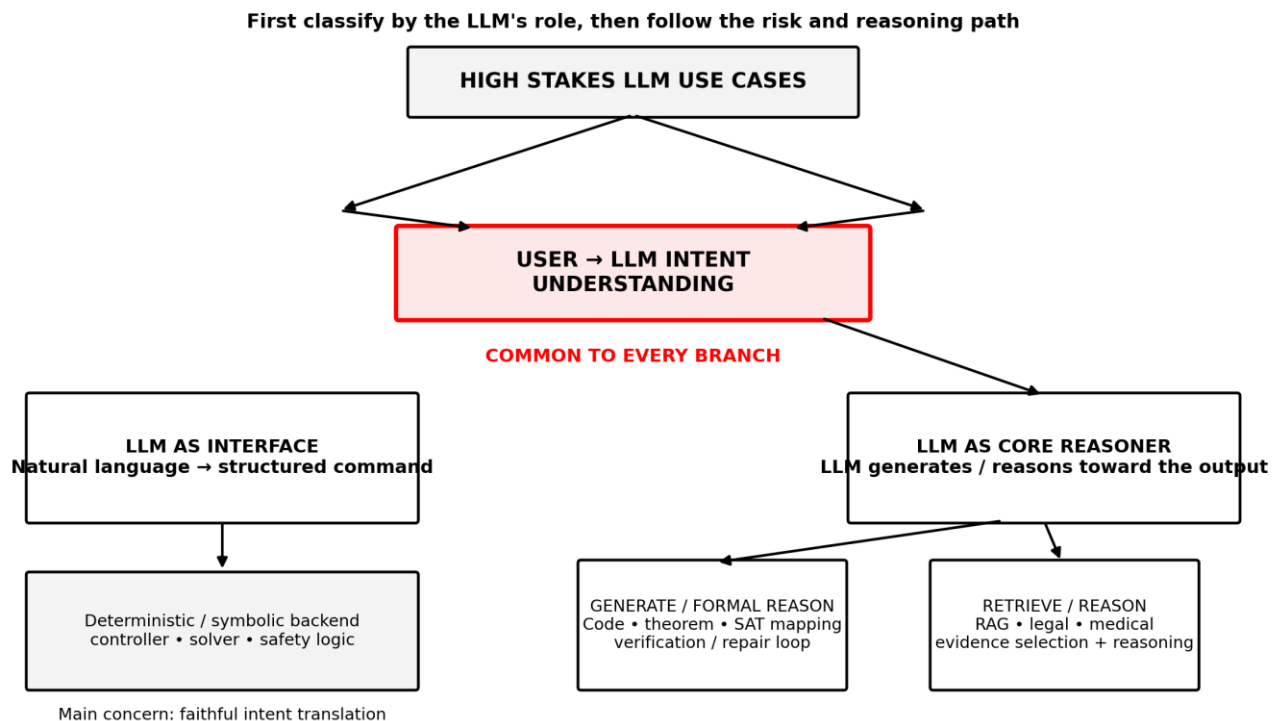
The first split

LLM as interface V/S LLM as core reasoner

In the interface case, the LLM mainly translates natural language into structured commands or parameters. The actual safety critical decision is still made by a deterministic controller, solver, policy engine, or other symbolic backend.

The key LLM problem is therefore faithful intent translation.

In the core reasoning case, the LLM itself contributes to the artifact or reasoning that determines the answer. This is where hallucination directly becomes part of the final result.



The high stakes examples I am interested in include safety critical and security critical software, kernel or low level code, protocol implementations, theorem proving, formal specification generation, static reasoning problems that are translated into SAT or SMT representations, legal reasoning, medical reasoning, and evidence selection in domains where missing or incorrectly selected information can materially change the result.

What happens once the LLM is the core reasoner?

Here I see two broad narratives. The first is a construct and verify path. The LLM generates something or reasons toward a formal object, such as code, a theorem proof, a formal specification, or a SAT or SMT formulation.

The second is a retrieval and evidence path. The LLM has to understand a complex case, identify what matters, retrieve the right evidence, and then reason over that evidence.

These two paths should not automatically receive the same verification framework. In the first path, we can often define properties of the generated artifact and use tests, static analysis, SMT, model checking, theorem proving, refinement, or a repair

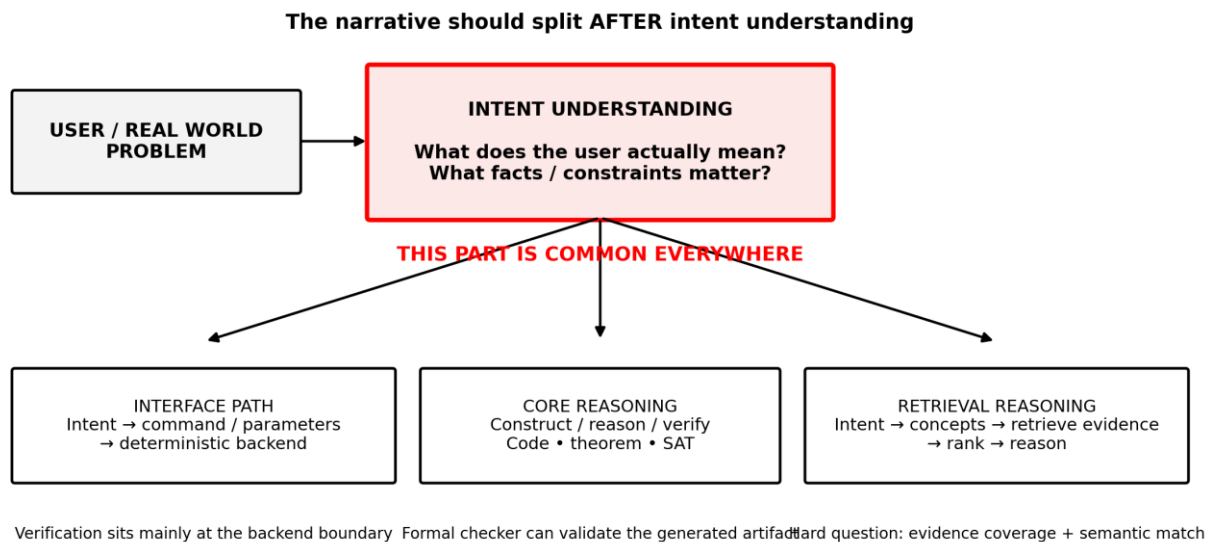
loop. In the retrieval path, the central problem is different. We need to understand whether the right concepts were extracted from the case and whether the retrieved evidence actually covers what the case requires.

5. Intent Understanding Is the Common Starting Point

What is common to all of these use cases before the narrative splits?

The common step is user to LLM intent understanding. Before the system can retrieve evidence, generate code, prove a theorem, formulate a SAT problem, or call a deterministic backend, it first has to understand what the user actually meant.

I want to make this part explicit because it is not just another stage in the pipeline. It is the common point of failure across the different applications.



The key observation

Intent understanding is common everywhere. The narrative changes after that point depending on what the LLM is being asked to do.

Why is intent understanding the main concern in the interface case?

If the LLM is only an interface, the downstream symbolic system can remain the correctness boundary. Once the LLM produces the correct command or parameters, the deterministic backend can enforce its own formal guarantees. So the major research concern for the LLM is whether the translation from natural language to the backend's representation is faithful.

Why does the problem become deeper when the LLM itself is reasoning?

Because there is no longer a deterministic component that completely replaces the LLM's reasoning. The LLM may have to construct an artifact, reason through a problem, choose proof steps, formulate a SAT or SMT problem, or select evidence from a knowledge base. The verification question therefore shifts from simply checking a backend result to checking whether the LLM's intermediate representations, retrieved evidence, reasoning, and final artifact are actually aligned with the original intent.

That is where I want the rest of the research discussion to branch.

For generation and formal reasoning, I want to study how the generated object can be checked and iteratively repaired.

For retrieval based reasoning, I want to study whether the problem representation and evidence selection can themselves be checked for coverage, consistency, semantic alignment, and missing information.

6. This Makes Problem Understanding the Upstream Bottleneck

The more I looked at different applications, the more this started to look like the same problem appearing in different forms.

- In code generation, natural language has to become a precise specification.
- In legal reasoning, a case narrative has to become structured facts and applicable rules.

- In medical reasoning, a patient narrative has to become the clinically relevant concepts and evidence.
- In theorem proving, a natural language problem has to become a mathematical statement.
- In model checking, a system description has to become a formal model and a set of properties.
- In retrieval, a complex request has to become the right semantic representation before the system can retrieve the right evidence.

So I do not think hallucination should be treated as one single failure mode. It can come from invented knowledge, retrieval failure, semantic misunderstanding, reasoning failure, formalization failure, generation failure, or specification failure.

Is the deepest common problem actually whether the LLM understood the problem correctly before we started verifying anything?

This is the question I find most compelling now. If the answer is no, then even a perfect verifier can give us a false sense of confidence.

7. The Retrieval Question

Can retrieval itself be checked rather than simply trusted?

This came up strongly when we considered legal and medical reasoning. A case does not necessarily contain the exact wording of the relevant law or medical evidence. **The system first has to understand what matters in the case, identify entities and relationships, determine which facts are important, and map those concepts to the relevant knowledge.**

That is more than putting documents into a vector database and retrieving similar chunks. The real question is whether the retrieved evidence satisfies the requirements imposed by the case.

Can we prove that retrieval is 100 percent correct?

I think that formulation is too strong because relevance is not naturally a binary mathematical property.

A better research question is **whether we can define necessary properties that a retrieval result must satisfy.**

- Required concepts are covered.
- Required entities and relationships are present.
- Mandatory constraints are represented.
- Important evidence categories are not missing.
- Retrieved rules are internally consistent.
- Contradictions are detected.
- Evidence actually supports the conclusions being drawn.
- Important assumptions are made explicit.

These are properties that are much easier to reason about rigorously than the vague question of whether one document is simply more relevant than another.

8. Knowledge Graphs Are an Implementation, Not the Research Question

Is the knowledge graph itself the main idea?

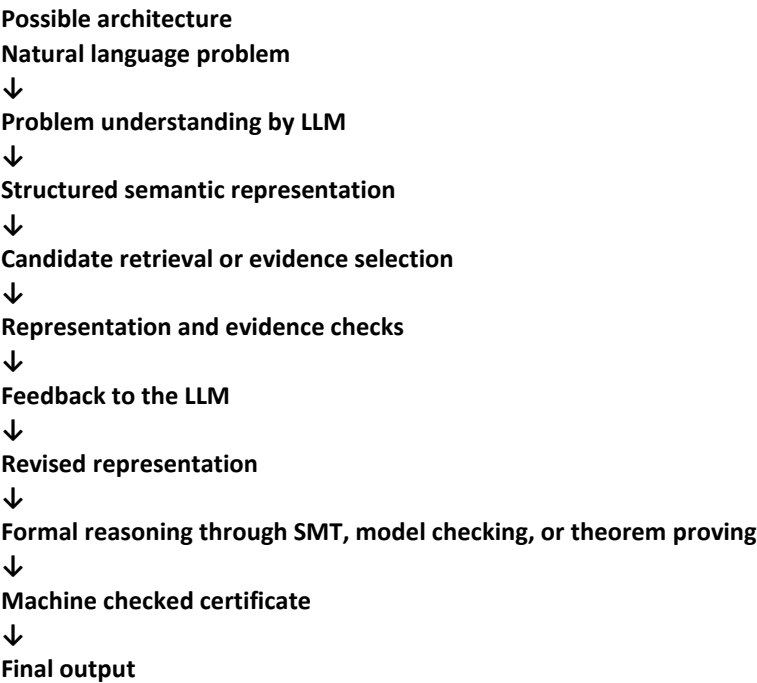
I do not think so. A knowledge graph could be a useful representation because it makes entities, relationships, rules, and concepts explicit. **But it is not the core research question.**

The deeper question **is whether an LLM can take a complex human problem and map it to the right semantic concepts, evidence, and constraints.** The underlying representation could be a knowledge graph, a database, a typed intermediate representation, or something else.

Where Can We Take This?

9. A Possible Research Architecture

The direction I am currently most interested in is **a verification aware semantic pipeline.** Instead of waiting until the final code or reasoning output to verify something, I would move some of the checking upstream.



This is where **I see a natural connection to PAT Agent.** PAT Agent already demonstrates the value of generating a formal artifact, checking it, using a counterexample as feedback, and asking the LLM to repair the artifact. **My question is whether the same loop can be moved earlier so that the semantic representation and evidence selection are also subject to structured checks.**

Can we use a formal or structured checker to identify missing evidence, contradictory assumptions, or semantic mismatches and then let the LLM repair its own interpretation?

That would turn the LLM from a one shot interpreter into an iterative interpreter that receives machine generated feedback about the representation it constructed.

10. Small Model Plus Verification (Benchmark Study)

Can formal verification replace some of the capability that we currently buy through model scale? (Ofc for some specific testcases)

I would compare a small model without verification, a large frontier model without verification, a small model with formal verification, and a large model with formal verification. I would not evaluate them only on final accuracy.

Metric	Why it matters
Functional correctness	Does the artifact actually work?
Formal assurance	What properties are machine checked?
Model size	How much model capacity is required?
Inference cost	How expensive is generation?
Verification cost	How expensive is assurance?
Latency	How practical is the complete loop?
Token usage	How much reasoning and repair is required?
Repair iterations	How often does verification force revision?
Failure severity	How bad are the remaining errors?

11. Final Conclusion?

Where is the actual research opportunity rather than just another implementation of LLM plus a formal tool?

I currently see four connected directions.

Direction A: Verification Aware Problem Understanding

Study whether an LLM can produce a structured representation of a problem that is not only syntactically valid but also faithful to the original intent. Use typed intermediate representations, constrained generation, multiple independent interpretations, scenario generation, or other mechanisms to expose semantic disagreement.

Can we detect when the LLM has produced a perfectly valid formalization of the wrong problem?

Direction B: Verification Aware Retrieval (Medical and Legal LLM)

Treat evidence selection as a structured object that can be checked for coverage, consistency, support, and missing requirements. The goal is not to prove that a retriever has found the one mathematically correct document. The goal is to verify properties that a sufficient evidence set should satisfy.

Can a checker tell the LLM that its current evidence set is incomplete or inconsistent and force another retrieval step?

Direction C: Adaptive Verification

Build a router that selects the verification mechanism according to the artifact and assurance target. Cheap checks could handle ordinary artifacts while SMT, model checking, or theorem proving is reserved for properties that genuinely require stronger guarantees.

Can we automatically decide how much assurance an artifact needs and which formal method is appropriate?

Direction D: Small Model Plus Formal Assurance

Measure whether formal verification can compensate for model capability. The key experiment is not just whether verification improves a model, but whether a smaller model with verification can approach the reliability of a much larger model without verification at a better total cost.

How much model capability can we replace with verification?